

STAGE 4: Flutter State Management

Flutter & Dart: From Zero to Expert — Stage Textbook 4 of 8

Goal: Move beyond local `setState()` and learn to share and manage state across an entire app.

Estimated time: 3-4 weeks

Prerequisite: Stage 3 (Flutter Basics)

4.1 The Problem `setState()` Doesn't Solve

`setState()` only rebuilds the widget that owns the state, plus its children. As soon as two unrelated widgets need the same data (e.g., a cart icon in the `AppBar` and a list of products on the body), passing state down through constructors becomes messy. This is called **prop drilling**, and it's the reason state management libraries exist.

4.2 InheritedWidget: The Foundation

Every state management library in Flutter (Provider, Riverpod, Bloc) is ultimately built on top of `InheritedWidget`.

```
class CounterModel extends InheritedWidget {
  final int count;
  const CounterModel({super.key, required this.count, required super.child});

  static CounterModel of(BuildContext context) {
    return context.dependOnInheritedWidgetOfExactType<CounterModel>(!);
  }

  @override
  bool updateShouldNotify(CounterModel oldWidget) => oldWidget.count != count;
}
```

Why it matters: `InheritedWidget` lets any descendant widget read data without it being passed explicitly through every constructor. You'll rarely write one by hand — but understanding it makes Provider and Riverpod feel far less magical.

4.3 Provider: The Practical Standard

Add to `pubspec.yaml`: `provider: ^6.0.0`

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class CartModel extends ChangeNotifier {
  final List<String> _items = [];
  List<String> get items => _items;

  void add(String item) {
    _items.add(item);
    notifyListeners(); // tells every listening widget to rebuild
  }
}

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => CartModel(),
      child: const MyApp(),
    ),
  );
}

// Reading the data anywhere below the provider:
class CartBadge extends StatelessWidget {
  const CartBadge({super.key});
  @override
  Widget build(BuildContext context) {
    final count = context.watch<CartModel>().items.length;
    return Badge(label: Text('$count'));
  }
}

// Updating it from anywhere below the provider:
ElevatedButton(
  onPressed: () => context.read<CartModel>().add('Apple'),
  child: const Text('Add to cart'),
);

```

Key rules:

- `ChangeNotifier` + `notifyListeners()` is the model layer; it holds data and announces changes.
- `context.watch<T>()` rebuilds the widget whenever the model changes.
- `context.read<T>()` reads the model once, without subscribing to changes (use inside callbacks like `onPressed`).

4.4 Riverpod: Provider's Compile-Safe Successor

Add to `pubspec.yaml`: `flutter_riverpod: ^2.0.0`

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

final counterProvider = StateProvider<int>((ref) => 0);

class CounterScreen extends ConsumerWidget {
  const CounterScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final count = ref.watch(counterProvider);
    return Scaffold(
      body: Center(child: Text('Count: $count')),
      floatingActionButton: FloatingActionButton(
        onPressed: () => ref.read(counterProvider.notifier).state++,
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Why consider it over Provider: Riverpod providers are not tied to `BuildContext`, are fully testable outside the widget tree, and catch missing-provider errors at compile time rather than runtime.

4.5 Bloc: State Management at Scale

Add to `pubspec.yaml`: `flutter_bloc: ^8.0.0`

```
import 'package:flutter_bloc/flutter_bloc.dart';

// Events
abstract class CounterEvent {}
class Increment extends CounterEvent {}

// Bloc
class CounterBloc extends Bloc<CounterEvent, int> {
  CounterBloc() : super(0) {
    on<Increment>((event, emit) => emit(state + 1));
  }
}

// Usage in the widget tree
BlocProvider(
  create: (_) => CounterBloc(),
  child: BlocBuilder<CounterBloc, int>(
    builder: (context, count) => Text('Count: $count'),
  ),
);
```

Why it matters: Bloc enforces a strict separation between **events** (what happened) and **state** (what the UI shows), which makes large apps with complex business logic easier to test and reason about. It has a steeper learning curve than Provider or Riverpod and is usually introduced once an app's logic outgrows simpler tools.

4.6 Choosing a Tool

Tool	Best for	Learning curve
<code>setState()</code>	Local, single-widget state	Trivial
Provider	Small-medium apps, shared state	Low

Riverpod	Medium-large apps, testability	Medium
Bloc	Large apps, complex business logic	High

4.7 Hands-On Project: Shopping Cart App

Build a small Flutter app that:

1. Shows a list of products (name + price) using `ListView.builder`.
2. Has an "Add to Cart" button next to each product.
3. Uses **Provider** with a `ChangeNotifier` cart model to track added items and total price.
4. Shows a cart badge in the `AppBar` with the item count, updating live from any screen via `context.watch()`.
5. Has a separate "Cart" screen reachable from the `AppBar` that lists everything added, using `context.read()` to remove items.

This project is the clearest possible demonstration of *why* state management exists: the badge, the product list, and the cart screen are all separate widgets sharing one source of truth.

4.8 Stage 4 Test

Question 1: What problem does state management solve that `setState()` alone cannot?

Question 2: What is the difference between `context.watch<T>()` and `context.read<T>()` in Provider?

Question 3: What does `notifyListeners()` do inside a `ChangeNotifier`?

Question 4: Name one concrete advantage Riverpod has over Provider.

Question 5: In Bloc, what is the difference between an "event" and a "state"?

Passing score: Get all 5 correct before moving to Stage 5.